

The Verified Core and the New Frontend: A Dependency-Light Csound Hearing-Aid Prescription Engine, Verified on the Bench and on a Manikin

David Akbari, Au.D., CCC-A^{1,2,3}

¹ American National Standards Institute / Acoustical Society of America S3/WG48

² International Electrotechnical Commission / Technical Committee 29

³ Cenerphone Technology Solutions, Inc.
dakbari@gmail.com

Abstract. In September 2024 the U.S. Food and Drug Administration authorized the first over-the-counter hearing-aid medical device software, turning a pair of consumer earbuds into a regulated medical device and making an audio signal-processing algorithm into a treatment whose behavior must be verified against a known output, not simply tuned by ear. This paper’s focus is to build the artifact the paradigm asks for: a dependency-light hearing-aid prescription engine written entirely as a Csound orchestra. From one editable row of audiogram thresholds it computes, at initialization time, a per-band NAL-R/RP insertion-gain target and a wide-dynamic-range compression (WDRC) law and applies them in an eight-band compressor — the whole device readable in a single sitting and portable, through libcsound, from a desktop rig to on-ear silicon. We verify it at two levels. On the bench, offline Csound renders establish that the core is deterministic to the sample (a bit-exact golden file), that its initialization-time prescription reproduces the published NAL-R formula exactly, and that its input/output behavior follows the prescribed compression law up to a fixed maximum-output ceiling. In situ, we couple the same orchestra to a manikin through an ordinary pair of AirPods — an interchangeable stand-in for any suitable transducer a finished product would use — and measure the delivered sound with a VeriFit 2 real-ear system against NAL-NL2 targets for a standard N3 hearing loss [25]. At a 65 dB SPL conversational input the aided response matches target with a root-mean-square deviation of 4.2 dB, excellent through the core speech region; the same measurement shows a high-frequency over-shoot consistent with a limitation the core’s own documentation predicted in advance. We frame the companion-interface and distributed co-processing ideas as hypotheses rather than results, recast azimuth-based source separation as a candidate alternative to directional-microphone beamforming for speech-in-noise, and place the whole exercise honestly inside the regulatory apparatus of IEC 62304, ISO 14971, and ISO 13485, with the road to clinical validation named but not yet walked.

Keywords: Csound API · software as a medical device · hearing aids · NAL · wide-dynamic-range compression · verification and validation · real-ear measurement · objective evidence · openFrameworks · distributed co-processing.

1 Introduction

On 12 September 2024 the U.S. Food and Drug Administration authorized the first over-the-counter (OTC) hearing-aid software device, intended to run on a widely available pair of consumer earbuds [1]. Once installed and customized to a user’s hearing sensitivity, the software amplifies and shapes sound for adults with perceived mild-to-moderate hearing loss. The authorization rested on a multi-site clinical study which found that users who self-fitted the device obtained benefit comparable to users fitted by a hearing professional [1]. What changed was not the transducers but a piece of digital signal processing (DSP) — and with that change the algorithm inherited the obligations of Software as a Medical Device (SaMD): it must do what it claims, provably, and its development must follow a defined software life cycle [2,3].

This is the crux, and it is easy to under-read. A hearing aid is not a wellness accessory; it is a treatment for a health condition. The moment a product tells the public that it amplifies sound for people with hearing loss, that public claim defines the product’s intended use, and the intended use defines the regulatory obligation. One does not escape that obligation by calling the software an “equalizer” or a “hearing enhancer.” The honest posture is the opposite of disclaiming: say plainly what the software is for, and then accept the

discipline that saying so entails. The OTC pathway sharpens the point, because it removes the licensed professional from the loop — the self-fitting user has no audiologist to catch an unsafe setting — so the safety a clinician would have supplied must instead be built into, and demonstrated by, the software itself.

This is a useful vantage point from which to look at Csound. The regulatory world for audio-based medical devices has arrived, by its own route, at an architecture the computer-music community has practiced for two decades: a computational core whose behavior can be tested in isolation, cleanly separated from the interface a human touches. Since the redesign that produced Csound 5 the system has been exactly this — a library, *libcsound*, with a stable public interface, over which any host may build a frontend [4,5]. There is a helpful analogy in the history of music itself. Common-practice notation let a composition be preserved, transmitted, and re-performed faithfully for centuries, independent of any one performer; a Csound orchestra is an executable in the same spirit — a self-contained, portable specification of a signal process. That durability is not a sentimental virtue. It is precisely the property a medical device’s configuration management and long-term maintenance demand: the ability to reproduce the exact processing, bit for bit, years later and on different hardware.

A word on vocabulary, because two professions use the same words differently and this paper lives at their intersection. In medical-software engineering, *verification* asks whether the software was built correctly, and *validation* asks whether the correct software was built — whether it meets the user’s real need [3]. In clinical audiology the operational meaning is narrower and, at first glance, inverted: matching real-ear output to a prescriptive target is routinely called *verification*, while establishing patient benefit by self-report or speech testing is *validation*. The two usages reconcile cleanly once aligned. An engineer’s bench render is verification; a probe-microphone match to a NAL target is *also* verification, now of the delivered acoustic result in situ; and only outcome in a person — did they hear better — is validation in either sense. This paper reports verification of both kinds, on the bench and on a manikin, and is explicit that clinical validation, on people, remains future work due in part to the requirements on handling human subjects which fall outside the scope of this paper.

The contribution is therefore concrete. We present a small, dependency-light Csound hearing-aid prescription engine that turns an audiogram into an eight-band wide-dynamic-range fitting; we verify it on the bench with reproducible objective evidence; and — the center of the paper — we verify the same orchestra in situ, on a manikin, by real-ear measurement against clinical NAL-NL2 targets. Around that spine we set the architectural argument: how Csound’s plumbing could translate to an openFrameworks host (Section 2); why a graphics framework is the natural home for a headless device’s companion interface, offered as a hypothesis rather than a claim (Section 3); the verified prescription core itself (Section 4); bench verification (Section 5) and manikin real-ear verification (Section 6); what a formal audit would still demand (Section 7); forward-looking enhancements on the verified core, including source separation as an alternative to directional microphones (Section 8); the path to clinical validation and commercialization (Section 9); the pedagogical dividend (Section 10); and a conclusion (Section 11).

2 The Csound Plumbing: A Dependency-Light Core with a Host on Top

The essential design move in Csound’s history happened during Csound 5: the engine — the orchestra compiler, the performance loop, and the built-in opcodes — was isolated into *libcsound*, a portable library whose only hard dependencies outside the C standard library are a sound-file library and a threading layer [4]. Anything needing a heavier dependency is pushed outside the core, supplied as a plugin or by the host. The result is a two-category world: plugins extend the engine, hosts embed it [5]. For a device engineer this is not merely tidy; it is the difference between a small, auditable trusted computing base and a sprawling one — a distinction Section 7 shows to be central to any medical-software audit.

A host drives Csound through a small, stable interface. It creates an instance, compiles an orchestra, and then repeatedly calls the performance function, which renders one control period of ksmps samples per call. Audio crosses the boundary through two buffers, spin (input) and spout (output); control values cross through named channels written and read with *chnset* and *chnget*. Nothing about this interface mentions a screen, a mouse, or a touch surface. The core computes; the host decides how a human addresses it. This is exactly the separation of concerns that lets the same DSP run unchanged on an ear-worn device and in a desktop verification rig.

2.1 Translating the Plumbing to openFrameworks

openFrameworks is a natural host for this design, especially to contemplate embedding Csound-based prototypes and algorithms into commercial software [16]. The openFrameworks audio system delivers a periodic callback, ofSoundStream’s audioOut, that must be filled with a block of output samples on the audio thread. Mapping the two worlds is almost mechanical: on each callback the host copies the requested frames into spin, advances the engine by the matching number of control periods, and copies spout back out to the device. Buffer sizes are chosen so the openFrameworks block is an integer number of ksmps periods, and the audio thread does nothing but pump the engine. This provides a convenient conceptual guide to translate to-and-from Csound-based prototypes to software that can run on iOS and Android with a professional quality-looking front-end interface.

The interface — the part that changes from project to project — lives entirely above this line. A controls window built with ofxGui, a bank of sliders, or a bespoke touch surface all resolve to the same thing: values pushed into Csound’s control channels between callbacks or re-implemented in pure C++. One discipline matters here: passed values must cross the audio/GUI boundary safely. In our hosts the bridge is a set of std::atomic parameters written by the GUI thread and read by the audio thread, so that no allocation, lock, or blocking call ever touches the callback; in Csound terms the same values become channel writes, and the engine sees only chnget. The same core, unchanged, can be reached from a desktop window today and a mobile touch screen tomorrow — the Mobile Csound Platform showed that libcsound ports intact to iOS and Android with only the host layer rewritten [6]. Figure 1 shows the layering.

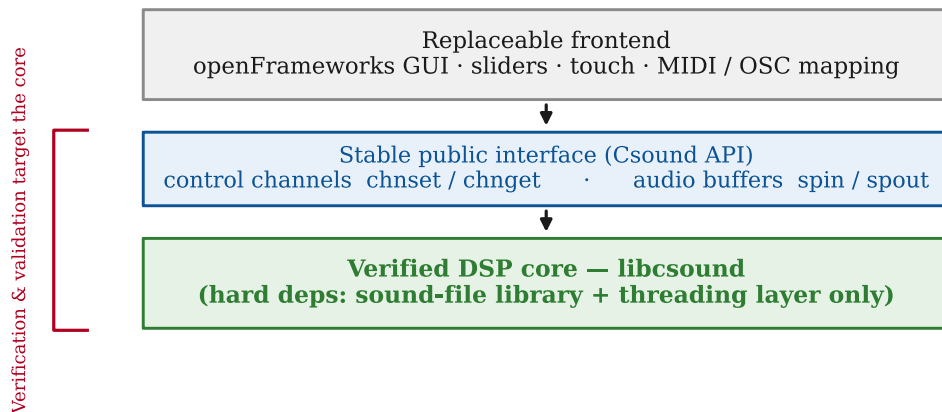


Fig. 1. The Csound plumbing as a layered host architecture. A verified DSP core exposes a stable interface — spin/spout buffers and control channels — to a replaceable openFrameworks frontend. Verification and validation target the core, below the interface.

3 A Headless Device and Its Companion: Distributed Co-processing

A fair objection follows immediately. Any audio library can pass blocks of samples to libcsound, and the medical devices in question — hearing aids, cochlear implants — are headless, resource-constrained, embedded systems with no display at all. Why host the core in openFrameworks, a framework built for heavy real-time graphics? The answer is that the display never was on the device, and increasingly is not expected to be. Contemporary hearing aids typically ship with a mobile companion application, and the trend on the instrument is toward fewer physical controls, not more, in favor of feature-rich haptic alternatives. Control has migrated to the phone. We take that migration as the opening for a design hypothesis rather than a finding: that the rich, visual companion interface is a large and under-explored space, and that a graphics-first framework is the right tool for exploring it.

The hypothesis has a name — *distributed co-processing* (Figure 2). The verified DSP core runs where it must, on the ear-worn device, inside a power budget measured in microwatts, as a low-overhead libcsound instance. The rich experience runs where it can afford to, on a phone or tablet with a GPU and a far larger battery, as an openFrameworks host. The two would communicate not by streaming audio but by exchanging a small amount of control over a low-rate link; the specific transport is deliberately left open, and the connectors openFrameworks already provides for it (ofxMidi, ofxOsc) are an implementation convenience, not part of the argument [24]. What matters is only that the boundary is a stable control interface, not a hardware assumption: the identical orchestra then runs on an earbud-class SoC, on a dedicated hearing-aid DSP, and on a phone, with only the host rewritten [6], and — the point this paper leans on — on a bench or manikin rig for verification.

The trade-offs are real and worth stating plainly, since they are what keep the proposal honest. On-ear power is scarce: a size-312 zinc-air cell holds roughly 160–170 mAh and a hearing aid draws on the order of a milliamp on average, giving several days to about a week of use — and wireless streaming, far more power-hungry than local processing, shortens that markedly. Keeping the interface off-device means the on-ear silicon spends its power budget on audio, not animation. We present the split as a plausible architecture to be evaluated, not as a measured system; its verification would be future work of exactly the kind Sections 5 and 6 model for the core.

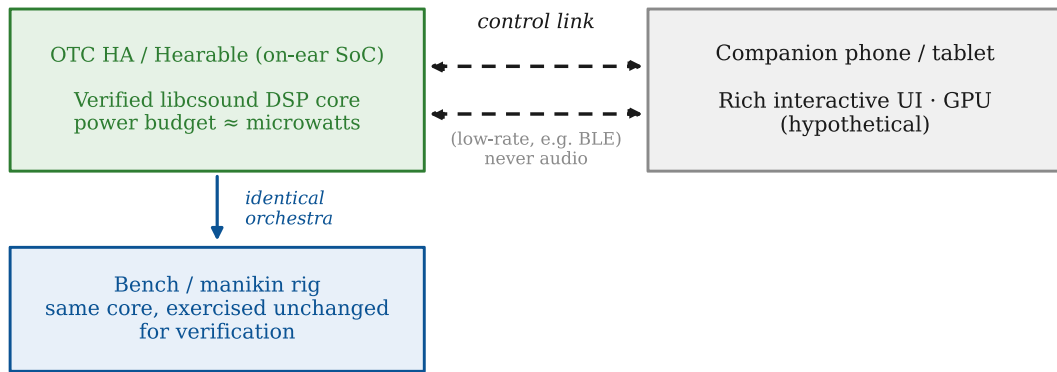


Fig. 2. Distributed co-processing, offered as a hypothesis. The resource-constrained hearable runs the verified libcsound DSP core; a companion phone or tablet would run a replaceable openFrameworks experience; the two would exchange low-rate control, never audio. The same core is exercised, unchanged, on the bench and manikin for verification.

4 The Verified Prescription Core

Wide-dynamic-range compression (WDRC) is the heart of a hearing aid: soft sounds must be amplified more than loud ones, so that a narrowed residual dynamic range is repacked into the listener’s reduced auditory area, and this must be done independently in several frequency bands because hearing loss is frequency-specific. The device at the center of this paper is a complete, if deliberately simple, expression of that idea as a single Csound orchestra. Its defining feature is that the entire fitting is *driven by the audiogram*: one edits a single row of eight threshold values, in dB HL, and the orchestra computes a per-band prescription — an insertion-gain target and a compression ratio — at initialization time, before any audio flows. The audiogram used throughout is the standardized N3 moderately-sloping loss (35, 35, 40, 45, 50, 55, 60, 65 dB HL at 250–6000 Hz; three-frequency average 42 dB HL).

4.1 Encoding the Prescription: NAL-R/RP with a WDRC Extension

The prescriptive rule is the National Acoustic Laboratories' revised linear procedure, NAL-R/RP [7,15], chosen precisely because it is a published, encodable formula rather than a proprietary table. For each band center it sets the target insertion gain

$$IG(f) = X + 0.31 * H(f) + k(f), \quad X = 0.05 * (H_{500} + H_{1000} + H_{2000})$$

where $H(f)$ is the threshold at f , the profile term X is one-twentieth of the three-frequency average, and $k(f)$ is the tabulated NAL frequency-shaping constant [7]. This linear target is then made nonlinear — turned into WDRC — by two transparent steps. First, the compression ratio in each band is made to rise with the degree of loss, $CR = \text{clip}(1 + 0.03 * H(f), 1, 3.5)$, so that more impaired bands restore loudness more aggressively. Second, the mid-level NAL-R target (referred to a 65 dB SPL input) is converted into the below-knee makeup gain a compressor actually applies, $G_{\text{soft}} = IG(f) + (65 - 50) * (1 - 1/CR)$, capped at 40 dB for feedback safety.

We are explicit about what this is and is not. It is a *transparent approximation* of the behavior of NAL-NL2 [8], not the proprietary NAL-NL2 (or NL1) formula itself. It deliberately omits two things NL1/NL2 does: the additional high-frequency gain roll-off, and the reduction of gain at severe losses that loudness-normalization imposes. This is a structural property of the formula, not a conjecture: because the core reproduces NAL-R/RP without NL2's high-frequency roll-off or severe-loss caps, its prescribed gain must sit above an NL2 target at the highest frequencies by construction. The orchestra's header states this in advance as an expected limitation, and Section 6 shows a high-frequency excess in the delivered response whose direction matches — with the caveat, developed there, that the delivered response also carries the transducer's own shaping. A limitation predicted from the source code, documented before measurement, and then seen in the delivered result is worth more to a regulated pipeline than a silent one.

Listing 1. The audiogram-driven prescription, computed once at initialization time. Editing the eight values in iHL re-derives the entire per-band fitting before any audio is processed, written in pure Csound with classic syntax. (Signal-chain excerpt in Listing 2.)

```
instr 1
;==== EDIT ONLY THIS ROW: threshold in dB HL per band ====
;(250 500 1k 1.5k 2k 3k 4k 6k) -- IEC 60118-15 N3
iHL[] fillarray 35, 35, 40, 45, 50, 55, 60, 65
icf[] fillarray 250,500,1000,1500,2000,3000,4000,6000
ik[] fillarray -17, -8, 1, 1, -1, -2, -2, -2 ; NAL-R k(f)

iX = 0.05 * (iHL[1] + iHL[2] + iHL[4]) ; 3FA profile term
iGain[] init 8 ; below-knee makeup gain (dB)
iCR[] init 8 ; per-band compression ratio
iKneeSPL = 50 ; WDRC kneepoint (dB SPL)
iRefSPL = 65 ; level the NAL-R target refers to

iN = 0
while iN < 8 do
  iThisHL = iHL[iN]
  iCRv = 1.0 + 0.03 * iThisHL ; ratio rises with loss
  iCRv = (iCRv < 1 ? 1 : (iCRv > 3.5 ? 3.5 : iCRv)) ; clip to [1, 3.5]
  iIGref = iX + 0.31 * iThisHL + ik[iN] ; NAL-R/RP target IG
  iGsoft = iIGref + (iRefSPL - iKneeSPL) * (1 - 1/iCRv)
  iCR[iN] = iCRv
  iGain[iN] = (iGsoft > 40 ? 40 : iGsoft) ; feedback-safety cap
  iN += 1
od
```

Evaluated at the N3 audiogram, the orchestra prints the fitting shown in Table 2. Because this table is nothing but the formula above evaluated at eight numbers, it is trivially traceable: a requirement (NAL-R/RP)

maps to an implementation (Listing 1) maps to a value (Table 2), and Section 5.2 confirms the values are reproduced exactly.

Table 2. The per-band prescription the orchestra computes at initialization time from the N3 audiogram: compression ratio and below-knee makeup gain. This is the Csound console log print, reproduced here verbatim.

Band	Centre (Hz)	Threshold (dB HL)	Comp. ratio	Makeup gain (dB)
1	250	35	2.05	+7.8
2	500	35	2.05	+16.8
3	1000	40	2.20	+27.8
4	1500	45	2.35	+29.8
5	2000	50	2.50	+29.8
6	3000	55	2.65	+30.6
7	4000	60	2.80	+32.5
8	6000	65	2.95	+34.3

4.2 The Signal Chain, and the Transducer as an Interchangeable Detail

The audio path is equally plain (Listing 2): a mono input is split by a fixed eight-band Butterworth filterbank, each band is compressed by `compress2` with the ratio from the prescription and lifted by the band's makeup gain, the bands are summed, and a final compressor plus hard limiter act as the maximum-power-output (MPO) stage. The core is monophonic — one instance per ear — runs at a 24 kHz sample rate with a 32-sample control period, and adopts the calibration convention that 0 dBFS corresponds to 100 dB SPL, so that a level in dBFS is simply the SPL minus 100. Hearing aids typically run at much lower sampling rates than CD-quality audio, to maximize the available battery budget. The orchestra's 24 kHz sampling rate reflects real hearing aid practice rather than a limitation.

Listing 2. Signal-chain excerpt: fixed Butterworth filterbank, per-band WDRC drawing its ratio from the computed table, per-band makeup gain, and an output-limiting MPO stage. Abbreviated for space; the complete orchestra along with the complete and tested unified .csd accompanies the paper.

```

aMic in ; mono; one instance per ear
iklo = -(100 - iKneeSPL) - 5 ; knee in dBFS (0 dBFS = 100 dB SPL)
ikhi = -(100 - iKneeSPL) + 3

a1 butterlp aMic, 354
a2 butterbp aMic, 500, 350
; ... bands 3-7 at 1k, 1.5k, 2k, 3k, 4k ...
a8 butterhp aMic, 5000

ac1 compress2 a1,a1, -90, iklo,ikhi, iCR[0], 0.005,0.080,0.005
; ... one compress2 per band, ratio = iCR[n] ...
ag1 = ac1 * ampdb(iGain[0]) ; per-band makeup gain
; ...
aMix = ag1 + ag2 + ag3 + ag4 + ag5 + ag6 + ag7 + ag8

aLim compress2 aMix,aMix, -90,-6,-1, 20, 0.001,0.050,0.005 ; MPO
aOut limit aLim, -0.99, 0.99
out aOut

```

One design decision deserves emphasis because it repairs an apparent contradiction. During prototyping and bench work the orchestra is run full-duplex through whatever transducer is at hand — in our case an ordinary pair of Apple AirPods, driven through the host's audio device. It would be a category error to read that as the deployment architecture: the AirPods are not the product, and the DSP claim does not depend on them. In SaMD terms the transducer is a component of the delivery path, interchangeable by design; the

verified artifact is the orchestra. On a shipping device the identical orchestra would be compiled into on-ear silicon behind the manufacturer’s own receiver. The reason to say this precisely is that it is exactly what the two-level verification below is built to demonstrate: the bench isolates the DSP from any transducer, and the manikin measurement then puts a real (if incidental) transducer back into the path and checks the delivered result. Figure 3 draws the whole chain, from audiogram to eardrum, with the two verification taps marked.

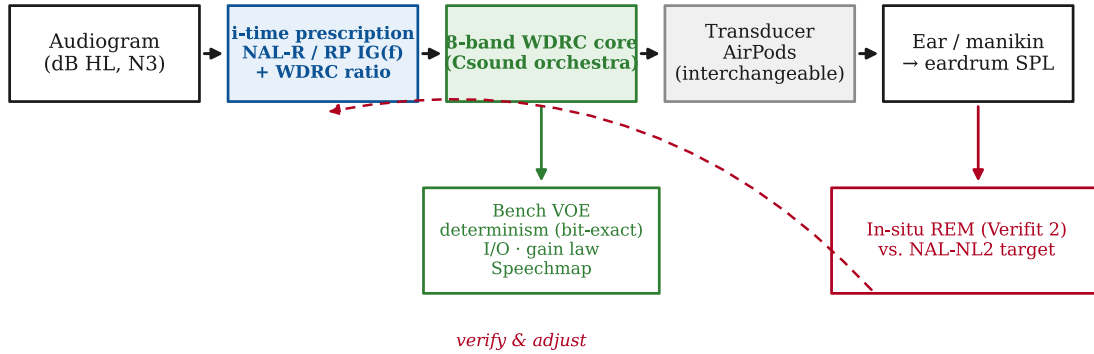


Fig. 3. The fitting-and-verification pipeline. An audiogram drives the initialization-time NAL-R/RP prescription, which parameterizes the eight-band WDRC core; the core is delivered through an interchangeable transducer (here AirPods) to the ear or manikin. Bench objective evidence taps the core in isolation; in-situ real-ear measurement taps the delivered result and compares it with the NAL-NL2 target, closing a verify-and-adjust loop.

5 Bench Verification with Objective Evidence

Verification must rest on objective evidence rather than impression. We use the term *verifiable objective evidence* (VOE) for the artifact class that satisfies this: recorded outputs that demonstrate a requirement is met and that an independent party can re-derive — a render, a hash, a log, a measurement, together with the script and inputs that produce them. Everything in this section and the next is VOE. The bench evidence was produced with Csound 6.18.1 performing offline renders and Python (NumPy/SciPy) performing the measurements.

5.1 Determinism: A Bit-Exact Golden File of the Device

The bedrock of verification is determinism: the same orchestra and input must produce the same samples every time, or no later comparison means anything. We rendered the device offline twice, in independent runs, driving it with a fixed calibrated stimulus, and hashed the output samples. The SHA-256 digests are identical (7d8dd989c86bf16e... 98308bf5) and the maximum sample-to-sample difference between the two renders is exactly 0.0. The render is therefore bit-exact and can serve as a golden file against which any future build is diffed, so that an unintended change to the device — a regression — announces itself as a changed hash. This is verification in its most literal form, and it is here performed on the actual medical-device orchestra, not a stand-in. This method is typical for hearing aid medical device software verification, SaMD or not.

5.2 Prescription Fidelity

The values Csound prints at initialization (Table 2) reproduce the NAL-R/RP formula of Section 4.1 exactly: recomputing $IG(f)$, CR, and G_{soft} independently in Python from the same eight thresholds reproduces every printed figure to the displayed precision (maximum discrepancy 0.0 dB). Trivial as arithmetic, this is the traceability link a life-cycle audit asks for: the requirement, the code, and the produced value agree, and the agreement is checked by a script rather than asserted.

5.3 The WDRC Law: Input/Output and Level-Dependent Gain

To characterize the dynamics, we drove the device with calibrated tones. Stepping a 2 kHz tone across input levels traces the input/output characteristic of Figure 4(a): the response is essentially linear below a knee near 48 dB SPL, compresses above it at roughly 2.5:1 — matching the ratio prescribed for that band — and is finally held by the MPO stage to a maximum output near 96 dB SPL, the safety ceiling that protects a wearer from over-amplified transients. Presenting the same device with tone complexes at three input levels (Figure 4(b)) recovers the two hallmarks of WDRC: the gain is frequency-shaped, following the loss, and it *decreases* as input level rises. These bench figures verify the core’s internal gain law and dynamics; they are expressed in the orchestra’s own 0 dBFS = 100 dB SPL convention and do not yet include any transducer or ear-canal acoustics, so the *absolute* match to an ear-referenced NAL target is not a bench question. That is the manikin’s job, and Section 6 turns to it.

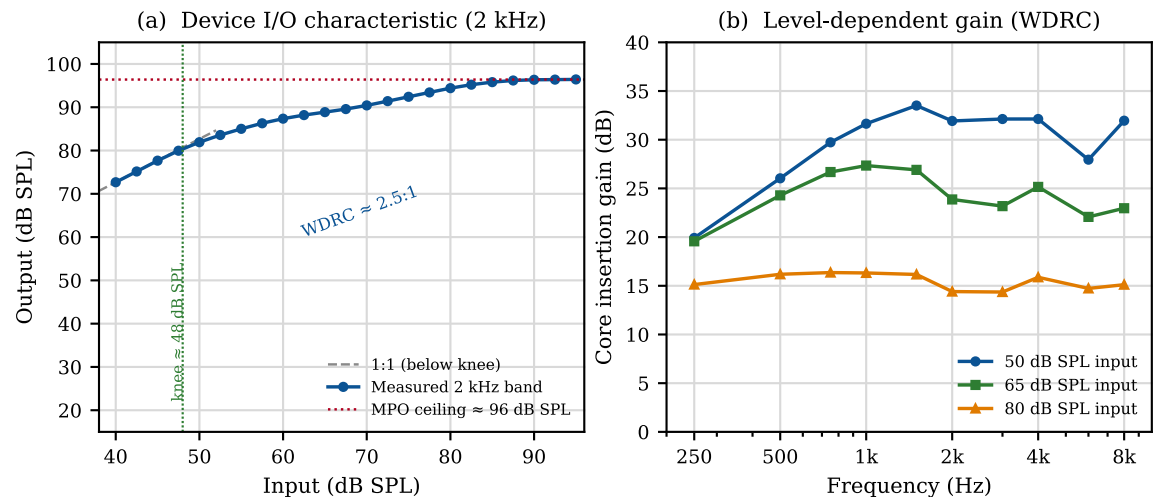


Fig. 4. Bench objective evidence for the device (offline Csound renders). (a) Input/output characteristic at 2 kHz: linear below a ~48 dB SPL knee, WDRC compression of about 2.5:1 above it, and an MPO ceiling near 96 dB SPL. (b) Core insertion gain at three input levels, showing the two hallmarks of WDRC — frequency-shaped gain that decreases as input level rises. Bench convention 0 dBFS = 100 dB SPL; no transducer in the path.

Table 3. Bench objective-evidence summary. Each stated specification is met by a recorded, re-derivable measurement produced by the released harness from fixed, deterministic inputs.

Property	Specification / prediction	Measured result
Determinism	identical output across renders	SHA-256 identical; max diff 0.0 (bit-exact)
Prescription fidelity	reproduce NAL-R/RP formula	exact; max discrepancy 0.0 dB
Compression ratio (2 kHz)	≈ 2.5:1 above knee	≈ 2.5:1 (slope 0.40)
Knee point	≈ 50 dB SPL	≈ 48 dB SPL
Maximum output (MPO)	bounded output	ceiling ≈ 96 dB SPL
Level dependence	gain falls as input rises	confirmed, all bands (Fig. 4b)

6 In-situ Verification on a Manikin by Real-Ear Measurement

Bench evidence shows the DSP is built correctly. A hearing device must also be verified where it matters — as delivered sound in an ear — because the transducer, the ear-canal acoustics, and the level-dependent interaction of speech with the compressor all lie between the orchestra and the eardrum. Real-ear measurement (REM) with a probe microphone is the clinical standard for exactly this [12], and it is the objective bridge this author is positioned to supply. In order to contemplate leveraging this software on human beings, it is necessary to ensure that there is a reasonable belief of efficacy when deployed. The clinician must have strong evidence to believe that the hearing aid intervention is likely to produce more benefit than simply not wearing a hearing aid at all. In this specific case, that prescribed gain is likely to match a known prescription target in a real-ear canal. The reason the manikin is used is precisely because it mimics the acoustic properties of the human body and has a realistic ear canal simulator [9, 11] thereby providing evidence of efficacy for the predicted SPL at the ear canal of a hypothetical human subject. This section is the paper’s center of gravity.

6.1 Method

The model device was fitted to the N3 audiogram of Section 4 and its output was measured on an Audioscan VeriFit 2 (serial 1059, software 4.20.1) in its on-ear speechmap mode, with the manikin’s ear terminated in an IEC 60318-4 occluded-ear simulator [11] — the standardized coupler that makes the measurement a repeatable in-situ proxy rather than a single subject’s ear. The KEMAR manikin was positioned in a soundfield where the manikin was placed at 0 degrees azimuth at a distance of approximately 0.5 meters from the emission source [26]. The orchestra was delivered through a pair of AirPods, as described in Section 4.2; the transducer is incidental to the DSP under test but represents a property of the whole signal delivery path. Prior to measurement, the AirPods’ adaptive features (Adaptive EQ, Adaptive Audio, and any active noise management) were disabled so that the transducer contributed only a fixed, passive response, removing time-varying processing as a confound. These settings on the AirPods were confirmed disabled in the device’s settings prior to measurement. Any other transducer mandates its own in-situ verification, which is one of the reasons Csound is an excellent choice for this exploration. The VeriFit 2 probe mic was positioned approximately 2 mm from the KEMAR’s simulated eardrum, confirmed via otoscopic examination. The stimulus is the International Speech Test Signal (ISTS), a language-neutral run-on speech signal [9], presented at 50, 65, and 80 dB SPL to probe soft, conversational, and loud inputs respectively using the calibrated VeriFit 2 to deliver the acoustic signal. The results reported are a single representative measurement; test-retest repeatability was within 0.5 dB at all tested frequencies including transducer reseating between trials. Targets are the VeriFit’s NAL-NL2 prescription for the same audiogram [8]; the analysis frequencies and the aided speech spectrum are reported on the SPL scale. Because the coupler represents the standardized average ear, this is in-situ *verification* in the clinical sense — a match-to-target measurement — and not yet individualized to a wearer’s own real-ear-to-coupler difference or venting. Showing target match on the manikin is a critical step to establishing the necessary efficacy to proceed to *validation* testing on human subjects by providing a reasonable expectation of prescription conformance, and therefore aided audibility for a given audiogram, which can simply be entered into the Csound orchestra.

6.2 Results

Figure 5 shows the aided speechmap. At the conversational 65 dB SPL input (panel a) the aided long-term speech spectrum sits on the NAL-NL2 target through the core speech region, with the whole 30th-to-99th-percentile speech band lifted above the SPLogram of audible thresholds and held below the measured uncomfortable levels. The instrument’s own root-mean-square deviation from target is 4.2 dB at 65 dB SPL, 7.0 dB for soft speech, and 5.6 dB for loud speech; recomputed over the primary 250–4000 Hz region the 65 dB deviation is 3.6 dB. By the ordinary clinical standard — a match within about 5 dB RMS across the speech frequencies — the conversational fit is good, and the fit across all three levels tracks the prescribed level-dependent shape (Figure 5b).

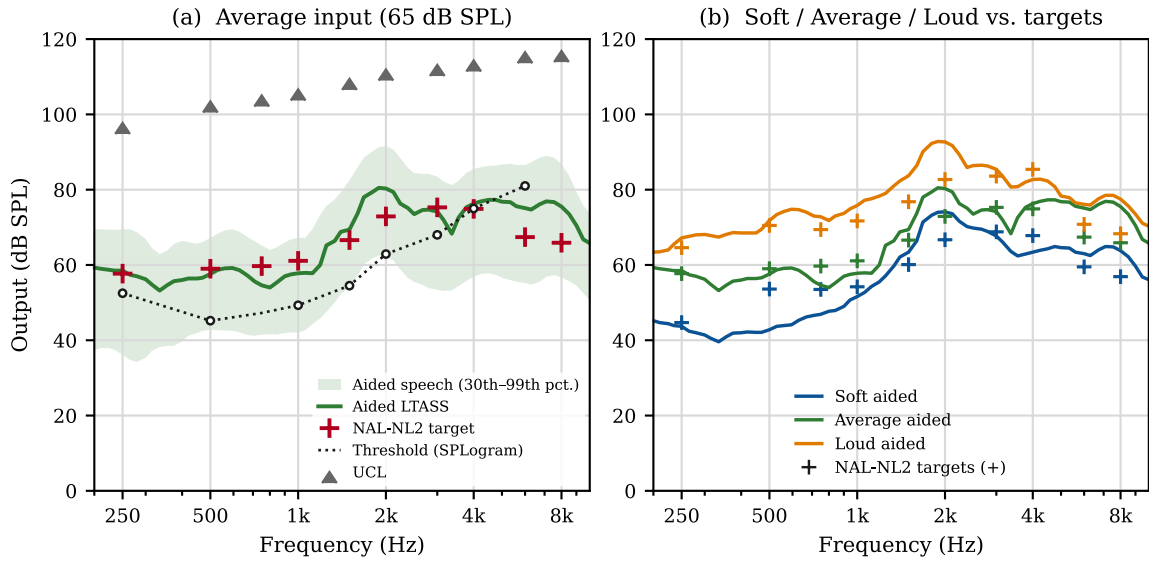


Fig. 5. Manikin real-ear (speechmap) verification on the Verifit 2, NAL-NL2, N3 hearing loss [25]. (a) Conversational 65 dB SPL input: aided long-term speech spectrum and 30th–99th-percentile band, NAL-NL2 target (crosses), threshold SPLogram, and uncomfortable levels (carets). (b) Soft/average/loud aided spectra against their respective targets, showing level-dependent tracking. Speech Intelligibility Index (SII) for 50, 65, and 80 was 8/32/71 in the unaided condition and 35/61/78 in the aided condition.

The posture the paper argues for while acknowledging the real limitations of the approach requires reporting where the fit departs from target, and the departures are informative (Figure 6). Three stand out. First, a consistent over-shoot at 6–8 kHz (about +8 and +10 dB at 65 dB SPL) — an over-shoot in the direction the core predicted in Section 4.1: the signature of a NAL-R-based fit that omits NL2's high-frequency roll-off and severe-loss caps. One caveat sets the strength of this claim precisely. The in-situ curve is the product of the core, the AirPods' transfer function, and the coupler. Because the AirPods' adaptive processing was disabled (Section 6.1), the transducer contributes only a fixed, passive response — so the high-frequency excess reflects the core's omitted NL2 roll-off, not time-varying device behavior. What still rides on the measurement is that one fixed transducer response, which the present setup does not separate out; characterizing it once, or re-measuring through a calibrated flat transducer, would isolate the core's contribution completely and is a clean next step the two-tier structure is built to accommodate. The claim is therefore calibrated, not triumphant: a limitation predicted from the source code, in writing, before any measurement, then observed in the expected direction and magnitude in the delivered result — with the one residual named rather than hidden.

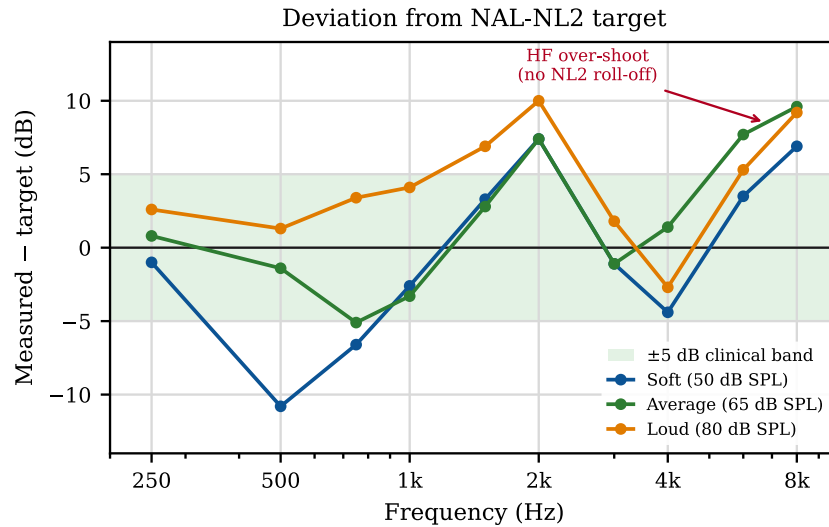


Fig. 6. Deviation of the measured aided response from the NAL-NL2 target at three input levels, with the ± 5 dB clinical band shaded. This deviation represents distance from target, not test-retest repeatability. The high-frequency over-shoot is in the direction predicted by the core's simplified NAL-R-based fit relative to NL2 (Section 4.1); as noted in Section 6.2, the in-situ deviation also includes the transducer path.

Two further objective outputs support the fit. The aided Speech Intelligibility Index rises with input level as expected — 35, 61, and 78 for soft, average, and loud speech — indicating restored audibility at conversational and loud levels for this moderate loss [10]. And the maximum-output measurement holds the device's loudest output below the measured uncomfortable levels across the frequency range, confirming that the MPO stage of Section 5.3 provides real headroom against over-amplification. Table 4 collects the match-to-target summary.

Table 4. Manikin real-ear match-to-target summary across the three ISTS input levels: RMS deviation from the NAL-NL2 target (Verifit-reported), the largest single-frequency deviation and where it falls, and the aided Speech Intelligibility Index.

Input (ISTS)	RMS to target (dB)	Largest deviation	Aided SII
Soft, 50 dB SPL	7.0	-11 dB @ 500 Hz	35
Average, 65 dB SPL	4.2	+10 dB @ 8 kHz	61
Loud, 80 dB SPL	5.6	+10 dB @ 2 kHz	78

The two levels of evidence now close on each other. The bench proved the DSP deterministic and its gain law correct in isolation; the manikin proves that, once a real transducer and a standardized ear are put back into the path, the delivered speech spectrum lands on a clinical prescriptive target at the conversational level that matters most — and that where it misses, it misses in directions the source code disclosed in advance. This is what it means to verify a Csound orchestra as a medical device rather than merely to audition it.

7 The Claim Is the Obligation: Life Cycle, SOUP, and the Limits of Architecture

Would this core realistically pass a formal medical-software audit? The honest answer is: not as it stands, but there is no architectural reason it could not, and the remaining gap is mostly documentation rather than code. IEC 62304, the software life-cycle standard the FDA recognizes, does not merely define verification and validation; it prescribes a whole life cycle [3] — planning, requirements, architectural and detailed design, implementation with unit verification, integration and system testing, and controlled release, with risk management, configuration management, problem resolution, and maintenance running alongside. The recurring theme is traceability: every requirement links to a design element, to the code that implements it, and

to the test that demonstrates it. Sections 4–6 are a small working example of exactly that chain, from an NAL requirement through an orchestra to a bench hash and a manikin curve.

A dependency-light core helps most visibly with one hazard the standard singles out: Software of Unknown Provenance (SOUP) — third-party components that enter the device without a full development record. Each item of SOUP must be identified, justified, and its risk managed. Because libcsound’s trusted computing base is small — in the limit the C runtime, a sound-file library, and a threading layer — there is very little SOUP to account for, and the audit surface shrinks accordingly. This is a genuine, architecture-level advantage, and it is the one this paper claims.

It is also the limit of the claim. A core/host split is good engineering; it is not, by itself, the SaMD regulatory apparatus, which is larger than any architecture. That apparatus includes ISO 14971 risk management across intended use and foreseeable misuse [18]; an ISO 13485 quality-management system governing the organization that builds and maintains the software [19]; and post-market surveillance that watches the device in the field. None of these is delivered by choosing the right architecture. Our position is therefore modest and, we hope, defensible: the dependency-light, testable core makes the software-engineering obligations — verification, validation, SOUP control, configuration management, reproducibility — tractable and demonstrable; it does not substitute for the risk management, quality system, and surveillance that surround them.

8 Forward-Looking Enhancements on the Verified Core

With a verified core in hand, each additional DSP capability can be treated as a fixed substrate and the design question becomes how a human should touch it. The scenarios below are deliberately forward-looking — grounded in the same architecture, but proposed rather than measured.

8.1 Azimuth-Based Separation as an Alternative to Directional Microphones

The hardest problem in amplification is the cocktail party: following one voice among many. The standard hearing aid answer is the directional microphone — a fixed or adaptive beamformer built from two or more microphones whose geometry favors sound from the front [15]. Beamforming works, but it is constrained by the few millimeters available for a microphone array on an ear-worn device, and its spatial selectivity is largely fixed by that geometry rather than chosen by the wearer. A software source-separation method offers a different handle on the same problem. ADress — azimuth discrimination and resynthesis — recovers sources by their inter-channel level differences, exposing frequency-dependent nulls across the stereo field, and locating a source at a recoverable azimuth [14]. On a verified core this suggests a steerable, user-directed complement to (or, for some listening situations, alternative to) a fixed beamformer: a companion display of the sound field on which the wearer taps the talker they want, setting the azimuth the separator emphasizes. The cancellation the algorithm performs becomes, at the interface, the gesture “hear that one.” The ADress algorithm separates by inter-channel level differences in a stereo mix. Hearing aid microphones do not present that unless there is bi-directional device communication, which means running ADress requires two channels working together. Porting ADress from stereo-mix separation to real binaural/array capture is itself a research problem. Nonetheless ADress represents a compelling new imagining of directional microphones in hearing aids made possible by the rich contributions of the many people who have contributed to Csound. We advance this as a hypothesis worth testing against directional-microphone benefit, not as a solved result; its value here is that it rides on exactly the same verified-core discipline the rest of the paper develops.

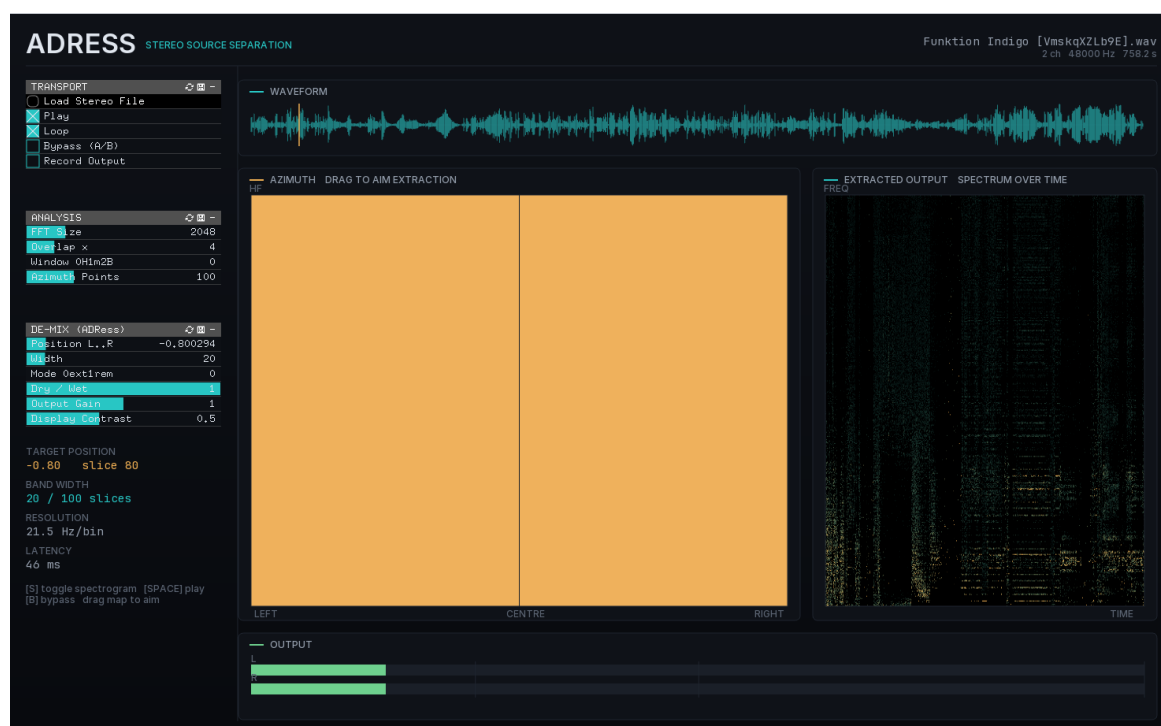


Fig. 7. The implementation of the working host ADDRESS, a steerable, user-directed complement to (or, for some listening situations, alternative to) a fixed beamformer directional microphone array. Presented here as a hypothesis worth testing against directional-microphone benefit. The source audio is “*Funktion Indigo*” (1969) by Gottfried Michael Köning.

8.2 Spatial Comfort, Self-Fitting, and Tinnitus as Possibilities

Two shorter possibilities follow the same pattern. A very lightly-set reverberation core could be repurposed not to add an effect but to restore a small, controlled sense of room for wearers whose spatial perception has been flattened by heavy compression and noise reduction — a “room-feel” control backed by a trusted core. While the “room-feel” working host is shown here, the possibilities extend far beyond this example. A calibrated synthesis core has two uses that have nothing to do with music, each an important aspect of a SaMD OTC hearing aid solution: a reproducible stimulus generator for a self-administered, self-fitting threshold task, which is exactly what an OTC device needs when no clinician is present, and a generator of the notched or shaped sound used in tinnitus sound therapy, which are all exciting hypothetical possibilities for a graphics-rich frontend split from the lightweight verified DSP core. In each case the point is that the sound is exactly specified and reproducible — the property Section 5.1 established.

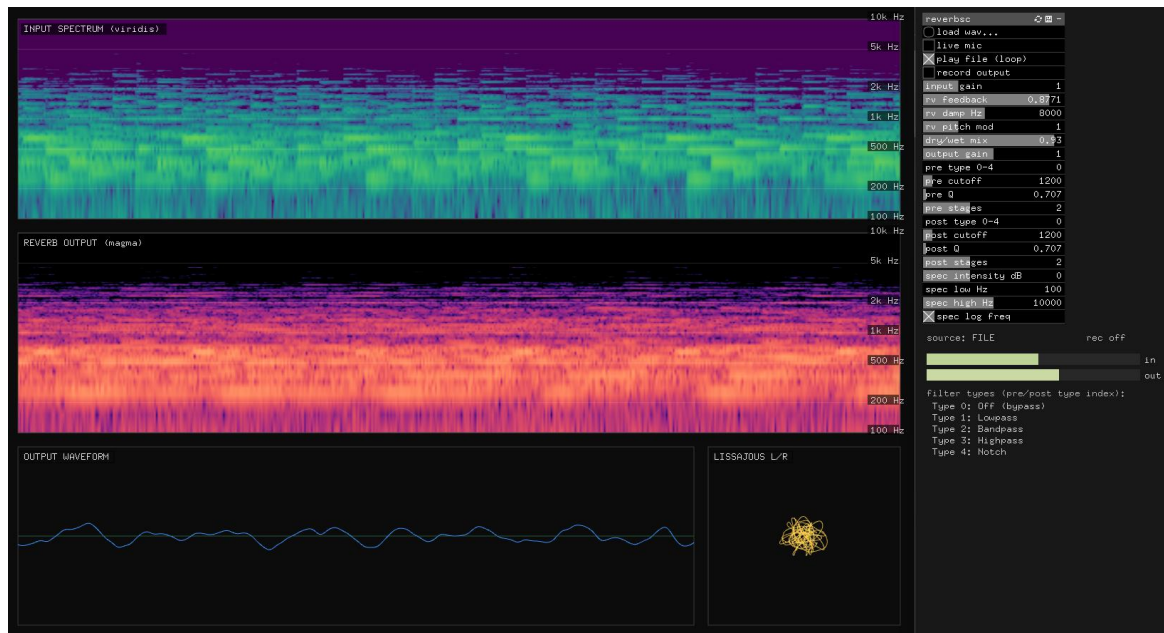


Fig. 8. The implementation of the working host ReverbSc, an 8 delay line stereo feedback delay network based on physical modeling scattering junction of 8 lossless waveguides of equal characteristic impedance to restore a “room-feel” for users whose spatial perception has been flattened by heavy compression and noise reduction. Presented here as a hypothesis worth testing in terms of user’s perception of benefit. The source audio is “*Chromatic Canon*” (1980/1983) by James Tenney.

9 Toward Clinical Validation and Commercialization

Verification, on the bench and on the manikin, shows that the device is built correctly and delivers its prescribed target on a standardized ear. It does not yet show that the device helps a particular person hear. That is a clinical question — validation in both the engineering and the audiological sense — answered on people with hearing loss using measures of outcome and benefit. The foundational idea is benefit as a difference: how a listener performs, or rates their own hearing, aided versus unaided. Validated self-report inventories quantify exactly this — the Abbreviated Profile of Hearing Aid Benefit [21], the Client Oriented Scale of Improvement [22], and the International Outcome Inventory for Hearing Aids [23] — and it is no accident that the FDA authorization that opens this paper turned on precisely such a measure, with self-fitting shown comparable to professional fitting [1,20].

The step between the manikin and the person is individualization. The measurement of Section 6 uses a standardized coupler; a real fitting would repeat the same real-ear procedure on the wearer, correcting the standardized coupler response by that individual’s real-ear-to-coupler difference and accounting for venting, before self-report outcomes are gathered. The apparatus for that — an individualized REM protocol feeding the verify-and-adjust loop of Figure 3 — is the natural next increment on the work reported here. Commercialization, finally, is what the discipline of Section 7 was for: standing up an ISO 13485 quality system, ISO 14971 risk management, and post-market surveillance [18,19], and binding the software’s semantic versions to the fitting formulae they implement, so that a clinician can know exactly which processing a given patient received. Framed this way, the standards are not a tax on creativity; they are the pathway by which an audio algorithm becomes a device that may honestly claim to help, and be believed.

10 A Pedagogical Core for New Frontends

There is an educational dividend to all of this. A Csound orchestra is a remarkably compact and inspectable specification of a signal process — often a handful of lines a student can read, run, and modify in one sitting

[17]. Coupled to the architecture above, the orchestra becomes a durable teaching kernel: because the core is small, portable, and testable, it can be held fixed while everything above it is reinvented. This inverts the usual difficulty of teaching interactive audio. Rather than asking a newcomer to build a synthesizer, an interface, and an audio pipeline at once, we hand them a verified core and pose a single question — how should a human touch this? — knowing the risky part, the DSP, has already been verified. The large companion-interface design space of Section 3 is, in pedagogical terms, an unusually inviting studio: the beginner works entirely above a floor that cannot give way.

11 Conclusion

Read through the lens of Software as a Medical Device, Csound turns out to be already shaped for the work regulation now demands of audio software. We have used that shape to build a real thing: a dependency-light hearing-aid prescription engine that turns an audiogram into an eight-band WDRC fitting, entirely as an inspectable Csound orchestra. We verified it twice over — on the bench, where it is deterministic to the sample and follows its prescribed compression law, and on a manikin, where the delivered speech spectrum lands on a clinical NAL-NL2 target at the conversational level with a 4.2 dB RMS deviation, missing chiefly in the high-frequency direction the source code predicted in advance. The architectural claims around it — the layered host, the distributed companion, the steerable separator — are offered honestly as hypotheses on top of that verified floor.

Two takeaways follow. First, a small testable core is the right foundation for the large, under-explored interaction design space that Software-as-a-Medical-Device OTC hearing aids open up: because the DSP can be fixed and trusted, designers and learners are free to reinvent the frontend above it without destabilizing the algorithm beneath. Second, the honest path from a clever algorithm to a device that helps a person hear runs straight through the discipline this paper has tried to take seriously — say plainly what the software is for, verify it with objective evidence on the bench and in the ear, validate it on the people it claims to serve, and maintain it as the treatment it is. The frontend is free to change because the core does not have to; and we can now show, on the bench and on the manikin, that the core does what it says.

References

1. U.S. Food and Drug Administration: FDA Authorizes First Over-the-Counter Hearing Aid Software. News release, 12 September 2024. www.fda.gov
2. International Medical Device Regulators Forum (IMDRF): Software as a Medical Device (SaMD): Key Definitions. IMDRF/SaMD WG/N10FINAL:2013 (2013)
3. International Electrotechnical Commission: IEC 62304:2006/AMD1:2015, Medical Device Software — Software Life Cycle Processes. IEC, Geneva
4. ffitich, J.: On the Design of Csound 5. In: Proceedings of the Linux Audio Conference. Karlsruhe (2005)
5. Lazzarini, V., Yi, S., ffitich, J., Heintz, J., Brandtsegg, Ø., McCurdy, I.: Csound: A Sound and Music Computing System. Springer, Berlin (2016)
6. Lazzarini, V., Yi, S., Timoney, J., Keller, D., Pimenta, M.: The Mobile Csound Platform. In: Proceedings of the International Computer Music Conference. Ljubljana (2012)
7. Byrne, D., Dillon, H.: The National Acoustic Laboratories' (NAL) New Procedure for Selecting the Gain and Frequency Response of a Hearing Aid. *Ear and Hearing* 7(4), 257–265 (1986)
8. Keidser, G., Dillon, H., Flax, M., Ching, T., Brewer, S.: The NAL-NL2 Prescription Procedure. *Audiology Research* 1(1), e24 (2011)
9. Holube, I., Fredelake, S., Vlaming, M., Kollmeier, B.: Development and Analysis of an International Speech Test Signal (ISTS). *International Journal of Audiology* 49(12), 891–903 (2010)
10. Acoustical Society of America: ANSI/ASA S3.5-1997 (R2017), Methods for Calculation of the Speech Intelligibility Index. ASA, New York
11. International Electrotechnical Commission: IEC 60318-4:2010, Electroacoustics — Simulators of Human Head and Ear — Part 4: Occluded-Ear Simulator. IEC, Geneva
12. Mueller, H.G.: Probe Microphone Measurements: 20 Years of Progress. *Trends in Amplification* 9(4), 153–171 (2005)

- 16 Akbari, D. (2026)
13. American National Standards Institute / Acoustical Society of America: ANSI/ASA S3.22, Specification of Hearing Aid Characteristics (2024)
14. Barry, D., Lawlor, B., Coyle, E.: Sound Source Separation: Azimuth Discrimination and Resynthesis. In: Proceedings of the 7th International Conference on Digital Audio Effects (DAFx-04). Naples (2004)
15. Dillon, H.: Hearing Aids, 2nd edn. Thieme, New York (2012)
16. openFrameworks. openframeworks.cc
17. Boulanger, R. (ed.): The Csound Book: Perspectives in Software Synthesis, Sound Design, Signal Processing, and Programming. MIT Press, Cambridge (2000)
18. International Organization for Standardization: ISO 14971:2019, Medical Devices — Application of Risk Management to Medical Devices. ISO, Geneva (2019)
19. International Organization for Standardization: ISO 13485:2016, Medical Devices — Quality Management Systems — Requirements for Regulatory Purposes. ISO, Geneva (2016)
20. Sabin, A.T., Van Tasell, D.J., Rabinowitz, B., Dhar, S.: Validation of a Self-Fitting Method for Over-the-Counter Hearing Aids. *Trends in Hearing* 24, 1–19 (2020)
21. Cox, R.M., Alexander, G.C.: The Abbreviated Profile of Hearing Aid Benefit (APHAB). *Ear and Hearing* 16(2), 176–186 (1995)
22. Dillon, H., James, A., Ginis, J.: Client Oriented Scale of Improvement (COSI) and Its Relationship to Several Other Measures of Benefit and Satisfaction. *Journal of the American Academy of Audiology* 8(1), 27–43 (1997)
23. Cox, R.M., Alexander, G.C.: The International Outcome Inventory for Hearing Aids (IOI-HA). *International Journal of Audiology* 41(1), 30–35 (2002)
24. openFrameworks addons: ofxMidi and ofxOsc. openframeworks.cc/documentation
25. Bisgaard, N., Vlaming, M., Dahlquist, M.: Standard Audiograms for the IEC 60118-15 Measurement Procedure. *Trends in Amplification* 14(2), 113–120 (2010)
26. American National Standards Institute / Acoustical Society of America: ANSI/ASA S3.35, Standard Method Of Measurement Of Performance Characteristics Of Hearing Aids Under Simulated Real-Ear Working Conditions (2020)

Appendix

```

<CsoundSynthesizer>
;=====
; audiogram_wdrc_hearing_aid.csd
; Audiogram-DRIVEN 8-band WDRC hearing aid.
; -> Edit ONE array of 8 dB HL numbers (iHL) and the orchestra
;   computes per-band insertion gain + compression ratio at
;   i-time, before any audio flows. Mono in/out; one instance
;   per ear.
;
; PRESCRIPTION CORE (encodable, published):
;   NAL-R / NAL-RP linear insertion-gain formula
;    $IG(f) = X + 0.31 * H(f) + k(f)$ 
;    $X = 0.05 * (H500 + H1000 + H2000)$  ; 3FA profile term
;   ...used as the target mid-level (~65 dB SPL) gain, then made
;   nonlinear (WDRC) by:
;   - a compression ratio that rises with degree of loss
;   - converting the 65 dB-SPL target into a SOFT-input makeup
;   gain given that ratio and kneepoint.
;
; This is a TRANSPARENT approximation of NAL-NL2's behavior,
; not the proprietary NL2 formula. Differences to expect vs.
; true NL2: NL2 adds extra high-frequency gain rollover and
; reduces gain at severe losses (loudness-normalization caps)
; that this simplified core does not. Verify any real fit with
; REM. See notes at the bottom to add an HF rollover / RP
; severe-loss correction.
;=====
<CsOptions>
-iadc:1 -odac:1 -rtaudio=coreaudio
; apple airpods in full duplex
</CsOptions>

<CsInstruments>
sr      = 24000
ksmps   = 32
nchnls  = 1
Odbfs   = 1

; CALIBRATION: compress2 is dBFS (0 dB = full scale).
; Assume 0 dBFS = 100 dB SPL, so dBFS = dB_SPL - 100.

instr 1
;=====
;==== EDIT ONLY THIS ROW: threshold in dB HL per band ====
;==== (250 500 1k 1.5k 2k 3k 4k 6k)      ====
iHL[] fillarray 35, 35, 40, 45, 50, 55, 60, 65
;
; IEC 60118-15 standard audiogram N3
;=====

; band centre freqs + NAL-R k(f) constants (parallel to iHL)
icf[] fillarray 250, 500, 1000, 1500, 2000, 3000, 4000, 6000
ik[] fillarray -17, -8, 1, 1, -1, -2, -2, -2

```

```

; ---- prescription computed at i-time -----
iX = 0.05 * (iHL[1] + iHL[2] + iHL[4]) ; 3FA term (H500,H1000,H2000)

iGain[] init 8 ; per-band soft-input makeup gain (dB)
iCR[] init 8 ; per-band compression ratio

iKneeSPL = 50 ; WDRC kneepoint we design around (dB SPL)
iRefSPL = 65 ; input level the NAL-R target refers to

iN = 0
while iN < 8 do
    iThisHL = iHL[iN]

    ; compression ratio rises with loss (WDRC loudness restoration)
    iCRv = 1.0 + 0.03 * iThisHL
    iCRv = (iCRv < 1.0 ? 1.0 : iCRv)
    iCRv = (iCRv > 3.5 ? 3.5 : iCRv)

    ; NAL-R/RP target insertion gain at iRefSPL
    iGref = iX + 0.31 * iThisHL + ik[iN]

    ; convert mid-level target -> below-knee (soft) makeup gain:
    ; above the knee, gain drops by (1 - 1/CR)*(input - knee)
    iGsoft = iGref + (iRefSPL - iKneeSPL) * (1 - 1/iCRv)
    iGsoft = (iGsoft > 40 ? 40 : iGsoft) ; feedback-safety cap

    iCR[iN] = iCRv
    iGain[iN] = iGsoft

    prints "band %d %5.0f Hz: HL %4.0f dBHL -> CR %.2f gain %+5.1f dB\n", iN+1, icf[iN], iThisHL, iCRv, iGsoft
    iN += 1
od

; ---- microphone input (mono; one instance per ear) ----
aMic in

; ---- shared WDRC dynamics + kneepoints (dBFS) ----
iatt = 0.005
irel = 0.080
ilook = 0.005
ithr = -90 ; gate floor, effectively off
iklo = -(100 - iKneeSPL) - 5 ; low knee ~ (iKneeSPL-5) dB SPL
ikhi = -(100 - iKneeSPL) + 3 ; high knee ~ (iKneeSPL+3) dB SPL

; ---- 8-band Butterworth filterbank (fixed crossovers) -----
a1 butterlp aMic, 354
a2 butterbp aMic, 500, 350
a3 butterbp aMic, 1000, 500
a4 butterbp aMic, 1500, 500
a5 butterbp aMic, 2000, 800
a6 butterbp aMic, 3000, 1000
a7 butterbp aMic, 4000, 1400
a8 butterhp aMic, 5000

; ---- per-band WDRC, ratio taken from the computed table -----
ac1 compress2 a1, a1, ithr, iklo, ikhi, iCR[0], iatt, irel, ilook
ac2 compress2 a2, a2, ithr, iklo, ikhi, iCR[1], iatt, irel, ilook
ac3 compress2 a3, a3, ithr, iklo, ikhi, iCR[2], iatt, irel, ilook

```

```

ac4 compress2 a4, a4, ithr, iklo, ikhi, iCR[3], iatt, irel, ilook
ac5 compress2 a5, a5, ithr, iklo, ikhi, iCR[4], iatt, irel, ilook
ac6 compress2 a6, a6, ithr, iklo, ikhi, iCR[5], iatt, irel, ilook
ac7 compress2 a7, a7, ithr, iklo, ikhi, iCR[6], iatt, irel, ilook
ac8 compress2 a8, a8, ithr, iklo, ikhi, iCR[7], iatt, irel, ilook

; ---- per-band makeup gain, taken from the computed table -----
ag1 = ac1 * ampdb(iGain[0])
ag2 = ac2 * ampdb(iGain[1])
ag3 = ac3 * ampdb(iGain[2])
ag4 = ac4 * ampdb(iGain[3])
ag5 = ac5 * ampdb(iGain[4])
ag6 = ac6 * ampdb(iGain[5])
ag7 = ac7 * ampdb(iGain[6])
ag8 = ac8 * ampdb(iGain[7])

; ---- recombine (complementary bands -> no /N) ----
aMix = ag1 + ag2 + ag3 + ag4 + ag5 + ag6 + ag7 + ag8

; ---- output limiter (MPO) + hard safety clamp ----
aLim compress2 aMix, aMix, -90, -6, -1, 20, 0.001, 0.050, 0.005
aOut limit aLim, -0.99, 0.99

    out aOut
endin

alwayson 1
</CsInstruments>

<CsScore>
f 0 z
</CsScore>
</CsoundSynthesizer>

```